Capítulo 3: Catálogo de metadatos de Hive

Contents

- Antecedentes: Apache Hive
- Unidades de información en el metastore de Hive
- Databricks y el catálogo de Hive

Antecedentes: Apache Hive

La tecnología Apache Hive fue desarrollada inicialmente por Facebook en al año 2007, y donado a la Apache Software Foundation en 2010 para convertirlo en un proyecto open-source. Podemos definir Apache Hive como:

> **Note**
>
> Sistema de Data Warehouse para manejar (leer, escribir, ETL, realizar reporting y análisis) datos almacenados de manera distribuida utilizando lenguaje SQL.

Hive proporciona un armazón SQL para construir tablas sobre ficheros de datos ya existentes, probablemente masivos, estructurados, que pueden estar en varios formatos como Parquet -formato más frecuente-, CSV, RCFile, ORC, ...) almacenados en diversas fuentes (la más frecuente era HDFS aunque posteriormente se añadieron otros sistemas de almacenamiento distribuido soportados por Hive, como Amazon S3). Las tablas son en realidad metadatos sobre estructura y tipos de datos, y localización física de los datos a los cuales apunta la tabla. Se introdujo cuando aún no existía Spark, como una manera de ganar abstracción y facilitar el uso de MapReduce de una manera menos árida y complicada de aprender como es SQL. La creación de tablas se efectuaba también mediante sentencias SQL de creación de tablas que incluían opciones indicando la ruta de los ficheros de datos que soportan a esa tabla.

Print to PDF

> **Note**
>
> Entre 2016 y 2019 muchas empresas montaron su lago de datos como tablas Hive definidas sobre ficheros Parquet en HDFS, seleccionaron Spark como motor y conectaron sus sistemas de BI a Hive. Es una arquitectura muy frecuente en sistemas legacy

Se trataba de una herramienta para acceder fácilmente a los datos usando exclusivamente lenguaje SQL, para uso principalmente por parte de analistas de Business Intelligence que conocen bien sus datos y el negocio, pero no dominan la programación, y para tareas como ETL, reporting y análisis de datos.

El lenguaje de consulta que utiliza se denomina HiveQL, que permite ejecutar prácticamente cualquier sentencia de SQL estándar y además tiene soporte para funciones a nivel de fila definidas por el usuario (User Defined Functions, UDF), y tipos de datos complejos.

> **Note**
>
> Con Spark es posible crear una UDF en cualquier lenguaje, registrarla permanentemente en el catálogo de Hive y utilizarla desde lenguaje SQL

Se utilizaba principalmente en modo OLAP (Online Analytical Processing), es decir, para realizar análisis en batch del informacional (histórico) de la empresa. Las consultas SQL no necesariamente se esperaba que fuesen interactivas: se programaban incluso ETL's con Hive en SQL. No es adecuado para OLTP (Online Transactional Processing) ya que no está pensado para manejo interactivo con respuestas rápidas ni para transaccionalidad. La ejecución de la consulta SQL la llevaba a cabo un motor de ejecución externo, intercambiable, que originalmente fue MapReduce pero después se podía configurar para apuntar a Spark o Apache Tez. Hive tiene un analizador SQL que traduce la consulta a jobs del motor de ejecución seleccionado.

Varias características de Hive lo diferencian de una Base de Datos pura, como la transaccionalidad (añadida después pero que penaliza mucho) o la imposibilidad de modificar datos, así como su filosofía de procesar la tabla como un todo, sin buscar la eficiencia en acceder a un registro concreto.

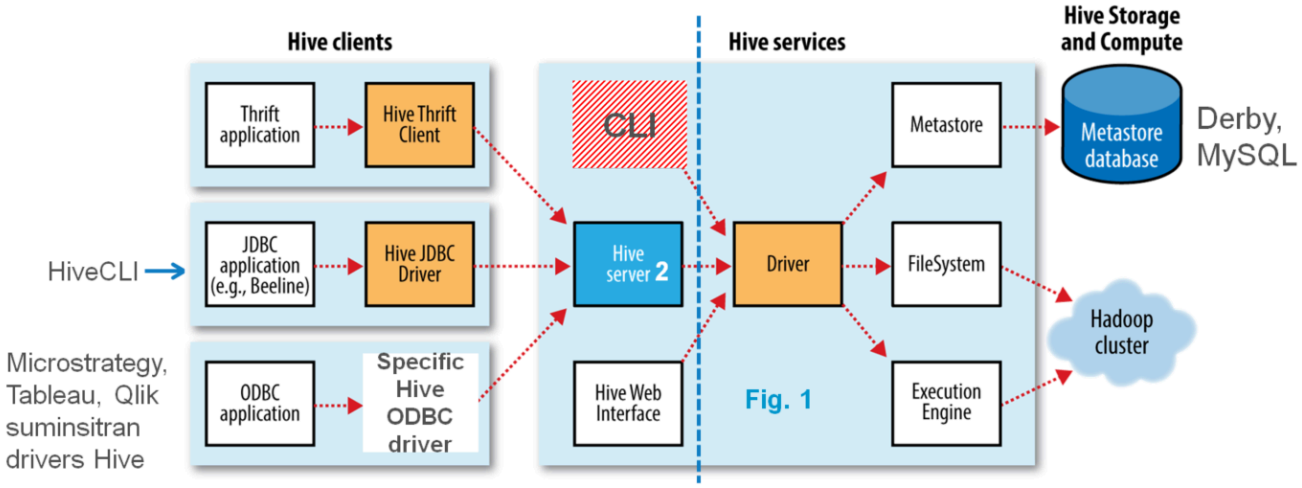
Relación entre Hive y Spark

La estructura de tabla se define creando metadatos en un formato concreto y almacenados en el catálogo de Hive, que habitualmente se almacena en una base de datos relacional convencional (por ejemplo MySQL para sistemas en producción, o Apache Derby como base de datos extremadamente ligera y compacta). A pesar de que no utilizaremos Apache Hive, el catálogo de metadatos creado por Spark se sigue llamando catálogo de Hive, aunque hay quien lo denomina “catálogo de Spark”.

> **Note**
>
> Apache Spark interactúa con el catálogo de metadatos de Hive (Hive metastore), ya sea con uno existente o creado por Spark, ya que entienden el mismo formato de metadatos.

> **Note**
>
> Spark incluye un analizador SQL para traducir SQL a jobs, y puede crear tablas a partir de ficheros de datos existentes, pero además permite definir vistas temporales sobre DataFrames en memoria. Por tanto, es posible definir un Data Warehouse utilizando Spark

Aunque ya no se utiliza, la manera en la que habitualmente se enviaban a Hive consultas SQL estaba basada en la arquitectura mostrada a continuación.



Arriba: maneras de utilizar Apache Hive (legacy)

Thrift Server 2 es un proceso demonio que está siempre levantado y escuchando en un puerto, a la espera de consultas. En la actualidad, es frecuente hacer algo similar pero sin Hive, utilizando exclusivamente Spark, ya que es posible conectarse mediante ODBC al

catálogo de metadatos de Hive creado por Spark "como si fuese una base de datos" en la que las consultas son ejecutadas siempre por Spark. Esto es posible gracias al Thrift Server que incorpora Spark, y que se detalla en https://spark.apache.org/docs/latest/sql-distributed-sql-engine.html

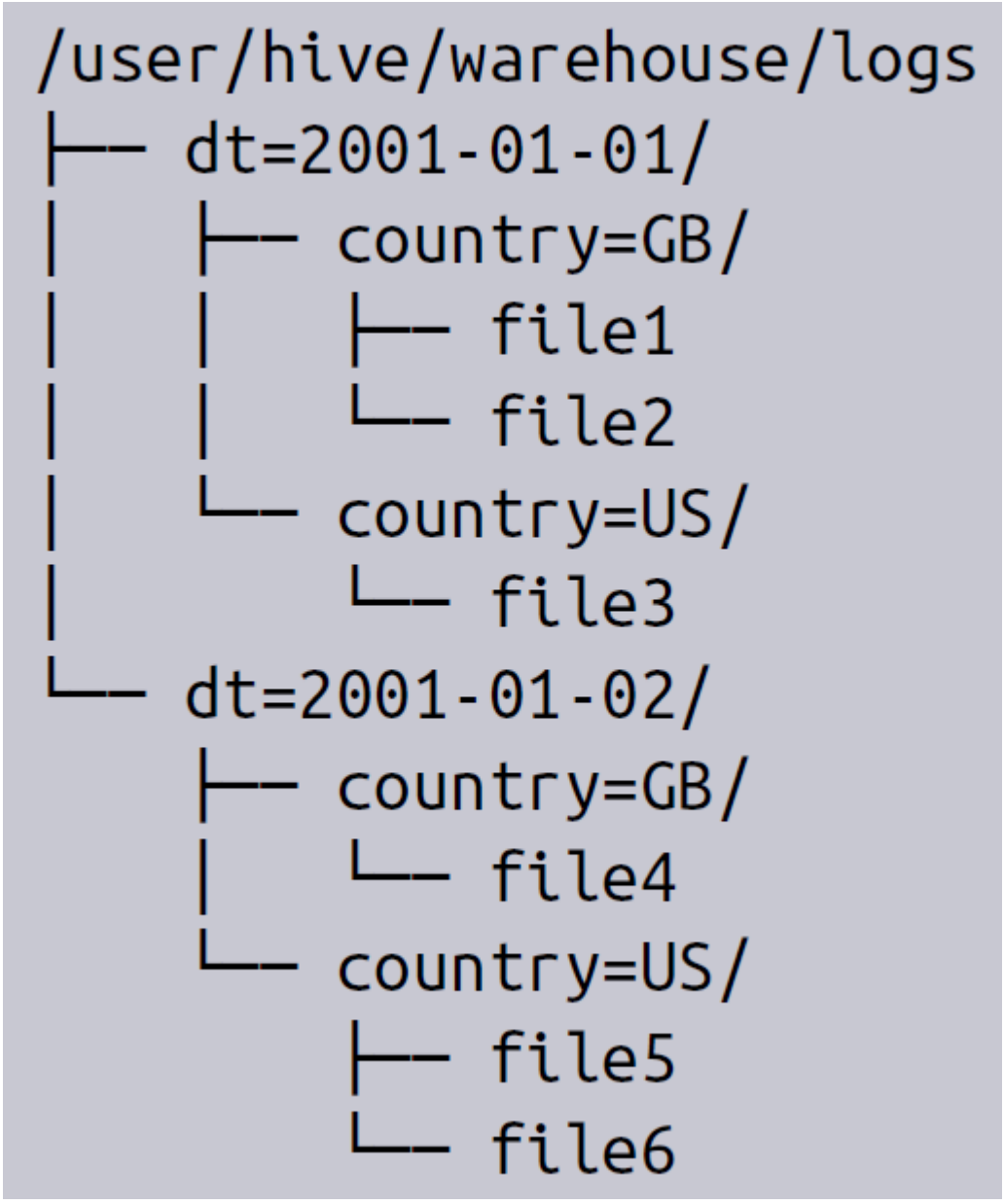
De hecho, es habitual ver Hive y también Spark como fuentes de datos (data sources) disponibles en las aplicaciones de Business Intelligence más populares, como Microstrategy, QlikSense, PowerBI, etc. En la actualidad, la conexión al Catálogo de Hive incluido en Databricks se lleva a cabo de forma similar, aunque Databricks ha anunciado conectores específicos para las herramientas más habituales de BI a través de su funcionalidad Partner Connect. Pueden encontrarse detalles en https://www.databricks.com/partnerconnect.

Unidades de información en el metastore de Hive

Estas son las distintas unidades de información que maneja Hive:

- Tabla: físicamente es un directorio de HDFS que almacena los datos de una tabla
- Partición: corresponde a un subdirectorio de HDFS dedicado al subconjunto de filas que tienen un cierto valor en una columna (la cual en realidad no forma parte de los metadatos)
- Bucket (compartimentos): grupos de filas de una tabla que no necesariamente comparten el valor de una columna, pero los valores de dicha columna han sido agrupados en unos pocos compartimentos (menos que los valores existentes). Hace más eficientes las consultas, incluyendo los JOIN si los atributos de JOIN están bucketizados en ambas tablas.

La siguiente figura muestra una tabla llamada logs particionada en base a dos de sus columnas.



Jerarquía de directorios donde se almacena la tabla logs particionada por las columnas dt y country

Tablas externas y tablas manejadas

Spark soporta los conceptos de tabla externa y tabla manejada que introdujo Hive:

- Tabla manejada (managed): Spark es totalmente dueño de ella. El borrado de la tabla implica borrado físico de los datos contenidos en la tabla. La ubicación de física de los datos para las tablas manejadas se configura en un parámetro de Spark y por eso no es necesario especificar ninguna ruta cuando queremos guardar un DataFrame de Spark como tabla manejada.
- Tabla externa (external): Spark sólo genera metadatos para ficheros de datos ya existentes. El borrado de la tabla solo borra metadatos del metastore de Hive, pero no

borra los datos físicos, puesto que estos ya existían desde antes de la creación de la tabla en sí, así que Spark considera que la tabla no le pertenece.

Puedes comprobar el funcionamiento de las tablas manejadas y externas en el notebook en el que manipulamos el catálogo de Hive. En él, primero creamos un DataFrame de Spark en memoria a partir de un fichero CSV, y procedemos a guardar el DataFrame:

- Primero, como como tabla (vista) temporal de Spark, que no guarda datos físicos sino que solamente crea en el metastore de Hive los metadatos necesarios para que la tabla pueda ser consultada mediante lenguaje SQL (incluso desde aplicaciones externas) durante esta sesión. Cuando cerremos el notebook y muera el proceso driver, la tabla desaparecerá y sus metadatos se borrarán del catálogo de Hive. Las vistas temporales son tablas no manejadas. Si hacemos un borrado de la tabla, no estamobs borrando datos ya que los datos ni siquiera han llegado a guardarse (a escribirse) en ningún almacenamiento persistente (ni HDFS ni ningún otro).

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.getOrCreate()
flightsDF = spark.read.option("inferSchema", "true").option("header", "true").csv('

flightsDF.createOrReplaceTempView("flights")  # esta vista desaparece cuando finali
                                              # La vista son solamente metadatos, d
consulta_df = spark.sql("select * from flights where Distance > 5000")  # el métodc
```

- Después, como tabla manejada, que guarda por un lado los datos físicamente en una ubicación física ya pre-configurada en una variable de entorno de Spark (que hace referencia a la ubicación donde se guardan los datos físicos desde Spark), y además crea los metadatos correspondientes en el catálogo de Hive. La tabla en este caso pertenece a Hive y se denomina manejada por Hive. Si hacemos un borrado de la tabla, estaremos borrando tanto los metadatos como los datos. En la plataforma de Databricks, el formato por defecto es Delta, y los datos se guardan dentro del contenedor de ADLS de la Storage Account privada que ha creado Databricks. En cambio en una instalación local en nuestro portátil, el formato habitualmente será Parquet y la ubicación por defecto será dentro de la carpeta spark-warehouse que Spark ha creado automáticamente.

```
# Guarda el DF como tabla manejada en una ubicación y formato ya configurados por d
# en la database "default" que es la única que existe por defecto. El concepto de d
# relación entre sí por la temática (por ejemplo, la database "financiero" donde se
# Es posible crear una nueva database con el comando spark.sql("CREATE DATABASE IF

flightsDF.write.option("mode", "overwrite").saveAsTable("default.vuelos")
```

- Por último, crearemos una tabla externa a partir de un fichero de datos CSV que ya existía en HDFS (nótese que pueden crearse tablas externas también a partir de otros sistemas de almacenamiento distintos a HDFS). Puesto que los datos que forman parte de la tabla ya existían con anterioridad a la creación de la tabla, Hive considera que dicha tabla no le pertenece y se denomina externa. Se crearán los metadatos y se escribirán en el catálogo de Hive, y esto será permanente incluso tras cerrar la sesión actual. Entre esos metadatos se encontrará almacenada la ubicación de los datos contenidos en la tabla. El borrado de la tabla no borrará los datos (el fichero CSV) sino que solamente borrará los metadatos que se crearon en el catálogo en el momento de crear la tabla.

```
# Opción 1: creamos tabla externa a partir de un fichero existente. La opción path
# Si el fichero es CSV, hay que indicar manualmente el esquema al crearla. Si es ur
# Para que no sea tedioso, vamos a guardar el CSV como Parquet y a crear la tabla c

flightsDF.write.mode("overwrite").parquet("C:/flights")
spark.sql("create table if not exists default.vuelos_externa using Parquet locatior
result_df = spark.sql("select * from default.vuelos_externa where Distance > 5000")
```

```
# Opción 2: salvamos el DataFrame en la ubicación que queremos, y sobre ella creamc
# Todo en una única operación. Spark entiende que es tabla externa porque hemos pue

flightsDF.write.mode("overwrite").option("path", "C:/flights_folder_ext").saveAsTab
consulta_df = spark.sql("select * from default.vuelos_ext2 where Distance > 5000")
```

Databricks y el catálogo de Hive

Tradicionalmente, Databricks ofrecía simplemente una interfaz gráfica al catálogo de metadatos de Hive para poder ver las tablas existentes y los metadatos de las tablas de manera gráfica, como vemos en la parte izquierda. Además, en 2022 introdujo la funcionalidad llamada Unity Catalog, que añade un nivel de abstracción al metastore, en el que se pueden compartir tablas y databases entre diferentes workspaces de Databricks (y por tanto, incluso entre diferentes empresas). Por ejemplo, ciertos usuarios pueden tener acceso solamente a un workspace con sus peculiaridades, su propio DBFS, etc, y a la vez pueden acceder a distintos catálogos que a su vez también pueden ser accesibles desde otros workspaces donde existen otros usuarios. El concepto de catálogo, como conjunto de databases, es transversal. Generalmente se asocia un catálogo a un ámbito de datos determinado, por ejemplo datos productivos, datos para pruebas, datos generados depurando otros datos, datos que son todos predicciones de modelos, etc. O incluso datos

de una empresa concreta perteneciente a un holding de empresas que, siendo distintas, comparten solamente ciertos datos o modelos.

Una tabla se nombra con 3 partes, nombreCatálogo.database.tabla. Además, Unity introduce muchas funciones adicionales, como una gestión totalmente centralizada del permiso de acceso a los datos, que antes se delegaba en Azure y se aplicaba a cada workspace por separado, mientras que ahora se centraliza la configuración deseada de cada workspace. También la posibilidad de ver el linaje del dato: para cada columna de cada tabla, Unity Catalog muestra la columna o columnas de la(s) tabla(s) a partir de las cuales está construida. Aprovechando el DAG de Spark, que intrínsecamente mantiene traza de todos los DataFrames (en realidad, de los RDD) de la aplicación, es posible traducir su estructura en un linaje del dato que se queda guardado.

